



My Network Scanner

MyNeS

You cannot secure what you cannot name.

Most home networks carry more devices than their owner can list. MyNeS finds every one of them, works out what it is from vendor, ports and protocols, lets you give it a name that means something, and tells you when something new appears.

Docker

Home Assistant add-on

CasaOS

MyNeS Mobile — the same engine, on the phone

App Store
iOS

Google Play
Android

github.com/fxerkan/my_network_scanner

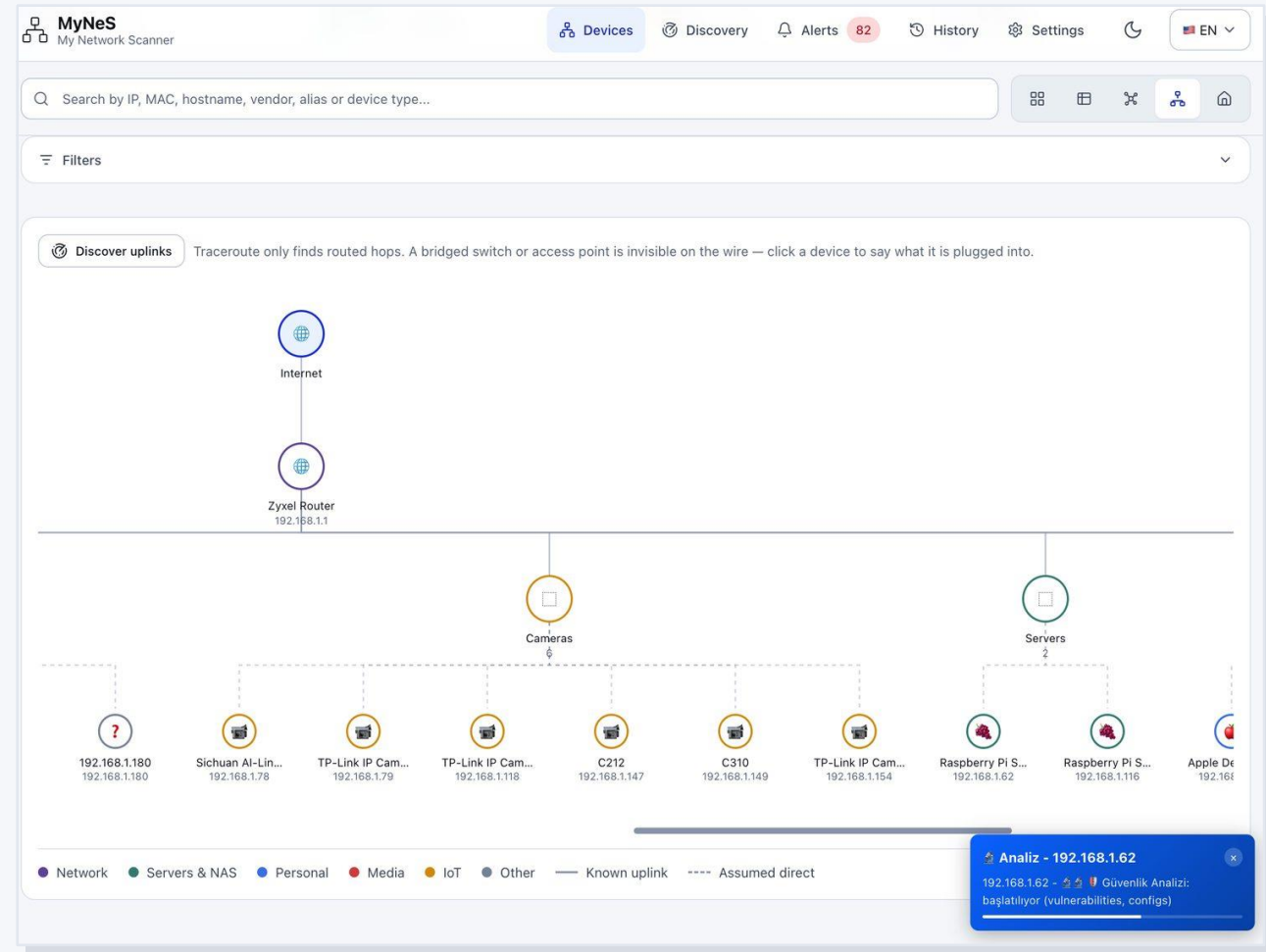
The screenshot shows the MyNeS web interface. At the top, there's a navigation bar with tabs: Devices, Discovery, Alerts (99+), Security, History, and Settings. Below the navigation bar, a dashboard displays four key metrics: 50 TOTAL DEVICES, 43 ONLINE DEVICES, 12 DEVICE TYPES, and 22 VENDORS. A search bar is located below the dashboard. Underneath, there's a 'Filters' section with checkboxes for 'Containers', 'No-IP devices', and 'Bluetooth / radio'. Below the filters, there are dropdown menus for 'Device Type', 'Status', 'Vendor', 'Alias', and 'Open Ports'. The main area displays a grid of device cards. Each card shows the IP address, device type, vendor, hostname, manufacturer, last seen time, and open ports. For example, the first card shows IP 192.168.1.1, Router, Zyxel Router, MAC 6c:11:14:14:14:14, Hostname EconetAPS, Manufacturer Zyxel Communications Corporation, Last Seen 10.08.2026 09:31:43, and Open Ports 80, 443, 53. Other cards include a Smart TV, an IoT Device, an Air Purifier, and a Smartphone.

Devices — every card is one real thing on your LAN

And it draws the shape of the network, not just a list.

The graph view traces uplinks, so a bridged switch or an access point that is invisible on the wire still shows up in the right place. Click a device and the whole path lights up.

- ARP + TCP sweep who answers
- IEEE OUI table which vendor, offline
- mDNS · SSDP · NetBIOS what it calls itself
- Ports + banners what it actually is
- Home Assistant over MQTT where it ends up



Graph view — uplinks traced, not guessed

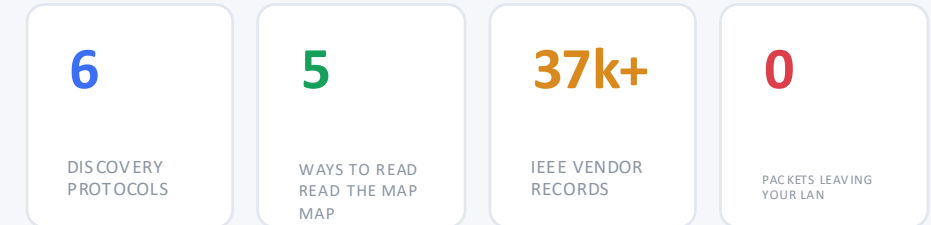


Half of a 2026 smart home has no IP no IP address.

Every scanner in the category walks the IP range. That was the whole network in 2015. Today the devices that matter most — the ones that watch, listen and unlock — often aren't on it at all.

- 1 Zigbee bulbs, plugs and sensors speak to a hub over 802.15.4, never to your router.
- 2 Z-Wave locks and thermostats are on their own 868/908 MHz mesh.
- 3 Bluetooth LE tags, scales and wearables advertise constantly — and hold no IP.
- 4 Matter devices announce themselves over mDNS in a way most scanners never

What MyNeS listens to



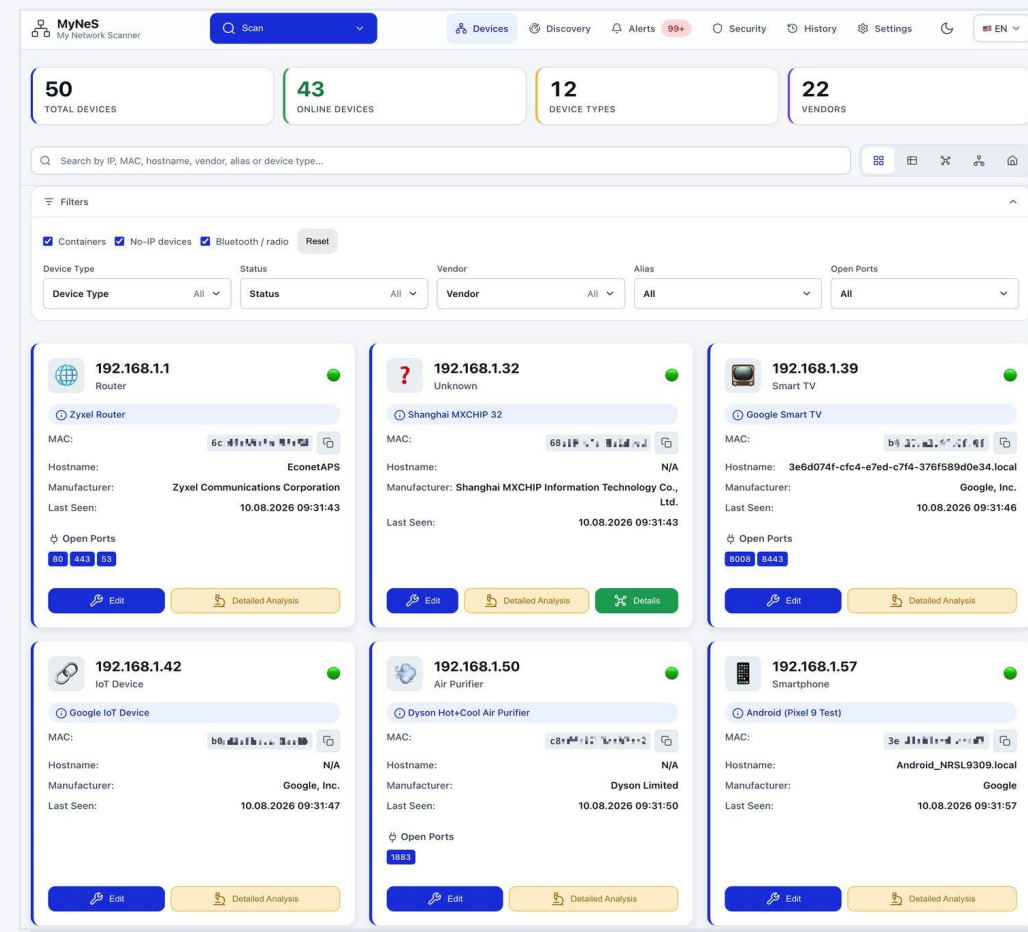
- An IP scan answers, “which addresses are up” . That is not the same question as “what is in my house”.
- MyNeS runs both: ARP + nmap for the IP side, and five more protocols for everything else.
- Zigbee and Z-Wave visibility is read from an existing Zigbee2MQTT / Z-Wave JS broker over MQTT — MyNeS does not talk to a radio itself.



Your family's friendly network scanner.

One self-hosted web app that discovers, identifies, names and watches every device on the LAN —and hands the whole inventory to Home Assistant.

- **Discover** — automatic range detection, raw ARP layer-2 sweep, 1000+ port service detection through nmap.
- **Identify** — IEEE OUI (37,000+ vendors), hostname patterns, service signatures, and an optional Identify with AI pass.
- **Name and own** — edit any device, add aliases, notes, types and trust levels; back the whole as JSON.
- **Watch** — scheduled re-scans, a diff against the last run, and alerts on what matters.
- **Integrate** — Home Assistant MQTT Discovery, InfluxDB, Prometheus, and a full documented
- **Own it** — AGPL-3.0, one container, runs on the Raspberry Pi you already have.





Six protocols, not one.

MyNeS listens to everything that announces itself, then merges it all into one device record.

- ARP (layer 2) — raw frames find devices that never answer a ping. Falls back to ping sweep + OS ARP cache OS ARP cache without privileges and tells you it did.
- mDNS / Bonjour — printers, NAS boxes, Chromecasts, HomeKit bridges.
- Matter — `_matter._tcp` announcements over mDNS.
- SSDP / UPnP — routers, smart TVs, cameras, consoles.
- Bluetooth LE — the devices with no IP address at all.
- MQTT — Zigbee2MQTT, Z-Wave JS and Tasmota retained topics (needs your existing broker).
- Docker — running containers, their virtual networks and IP assignments, nested under the them.

The screenshot shows the MyNeS web interface. At the top, there's a navigation bar with 'Devices', 'Discovery', 'Alerts' (82), 'History', 'Settings', and a language dropdown (EN). The main section is titled 'Protocol discovery' with a subtitle 'Beyond ARP and nmap: mDNS/Bonjour and Matter, SSDP/UPnP, Bluetooth LE, and Zigbee via MQTT.' There are buttons for 'Save to devices' and 'Run sweep'. Below this, there's a 'Protocols' section with a 'Sweep time' dropdown set to '8s'. It displays six protocol cards: Bluetooth LE (8 found, ready), DHCP (passive) (0 found, unavailable), mDNS / Bonjour (5 found, ready), MQTT (0 found, unavailable), ONVIF / RTSP (2 found, ready), and SSDP / UPnP (2 found, ready). Each card lists the types of devices it can find. Below the protocols section is a 'Discovered devices' section with a search bar and a table of 10 devices.

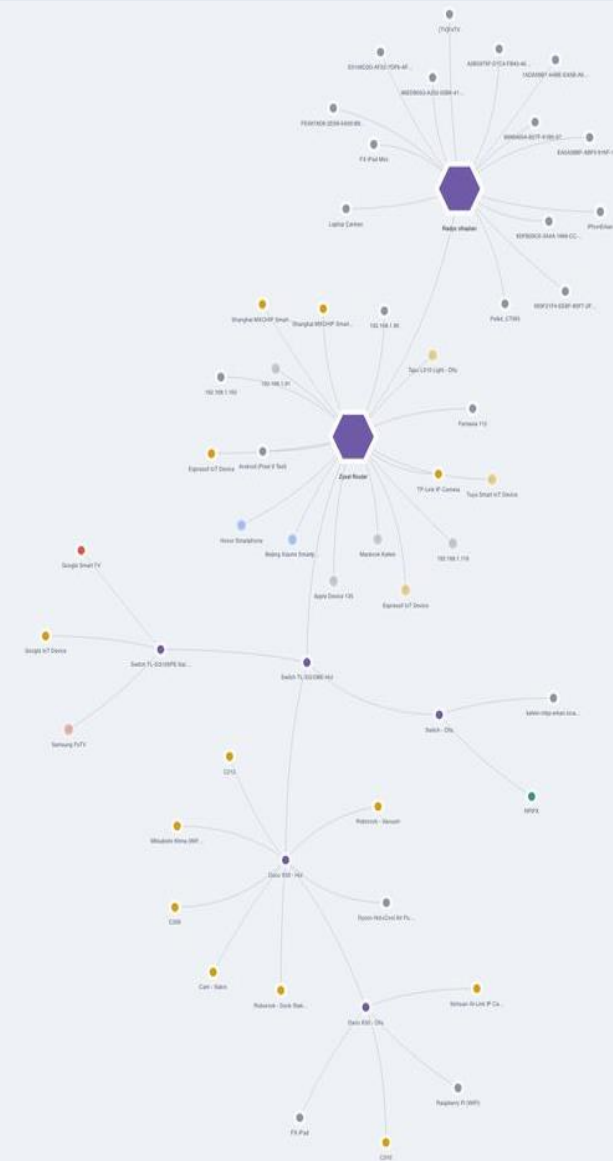
DEVICE	ADDRESS	TYPE	SEEN VIA	SERVICES
376f589d0e34.local Google	192.168.1.39	Media Player	mdns ssdp	Android TV Google Cast UPnP dial +5
7af...local	192.168.1.62	Server	mdns	Home Assistant HomeKit SMB Share Device Info +1
mbp-erkan.local	127.0.0.1	Media Player	mdns	Apple Companion net-assistant VNC SSH +4
iPad-Mini.local	192.168.1.188	—	mdns	Apple Companion
eWeLink...local	192.168.1.89	—	mdns	ewelink
EconetAPS Econet Technology, Corp.	192.168.1.1	Router	ssdp	UPnP WFAWLANConfig Layer3Forwarding WANCommonInterfaceConfig +2
C212	192.168.1.147	IP Camera	onvif	ONVIF RTSP
C200	192.168.1.154	IP Camera	onvif	ONVIF RTSP
B3E-1...		Bluetooth Device	ble	BLE



A device list is not a network.

The same inventory, read five ways — switched from one control in the toolbar.

- Card and Table — detail, bulk editing, sortable and reorderable columns.
- Graph — the whole network at a glance, with subnet gateways at the centre. A host's Docker bridge networks group *under* that host, not as peer LANs. peer LANs.
- Topology — what is plugged into what: Internet → router → switch/AP → group → device
- Floor plan — drag each device onto a plan of your actual home.
- All five read the same store, so a rename in one is a rename everywhere.

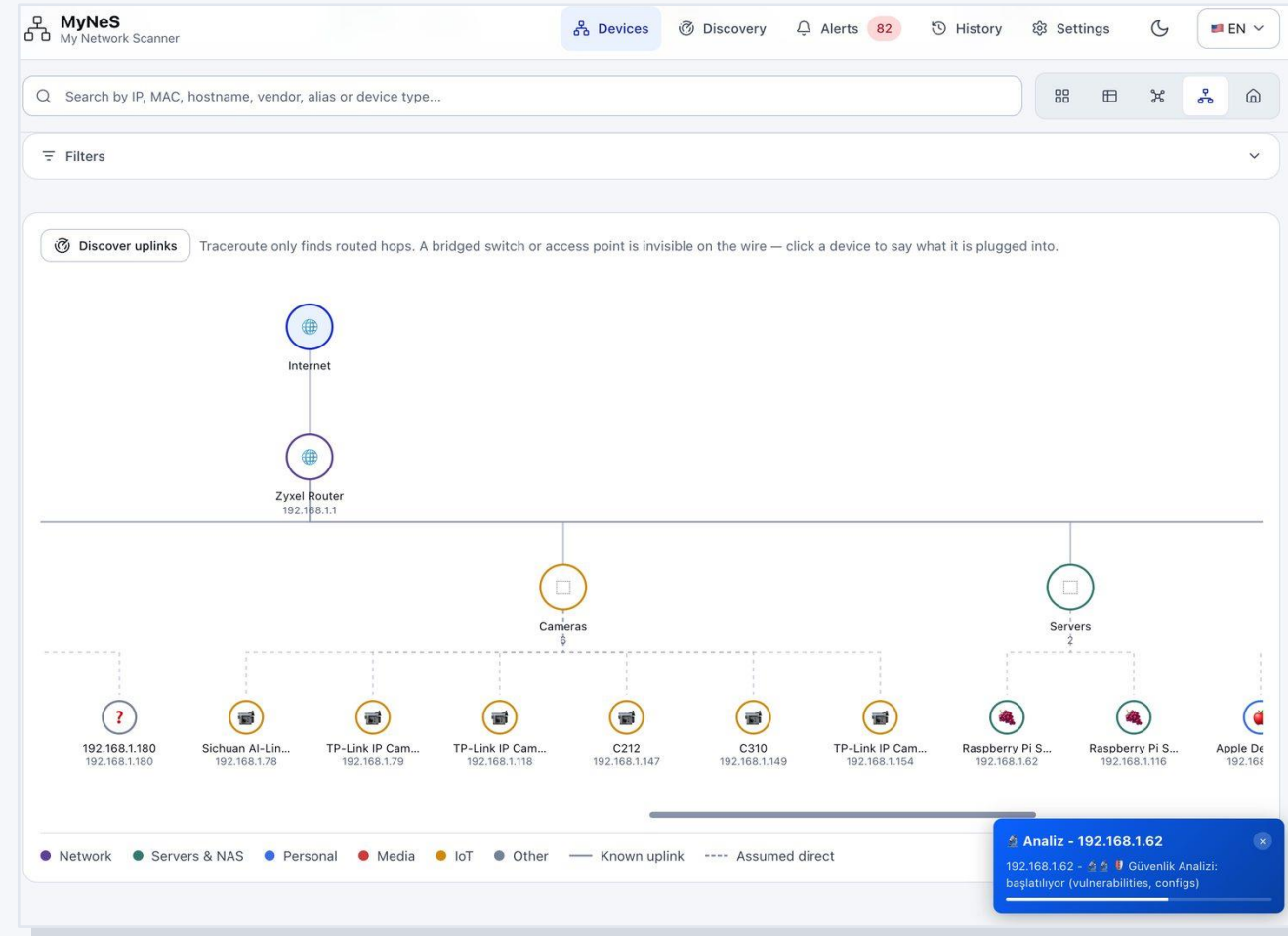




From cable path to living room.

Two views nobody else in this category ships.

- **Topology** answers the question a list cannot: *what is this device actually connected connected through?* Useful the moment a switch or an access point misbehaves. misbehaves.
- **Floor plan** puts the inventory where you actually live — drag a camera hallway, a plug to the kitchen.
- A device in the wrong room on the plan is usually a device you forgot you owned.
- Both views are built from the same scan data — no extra configuration, no no separate diagram to maintain.

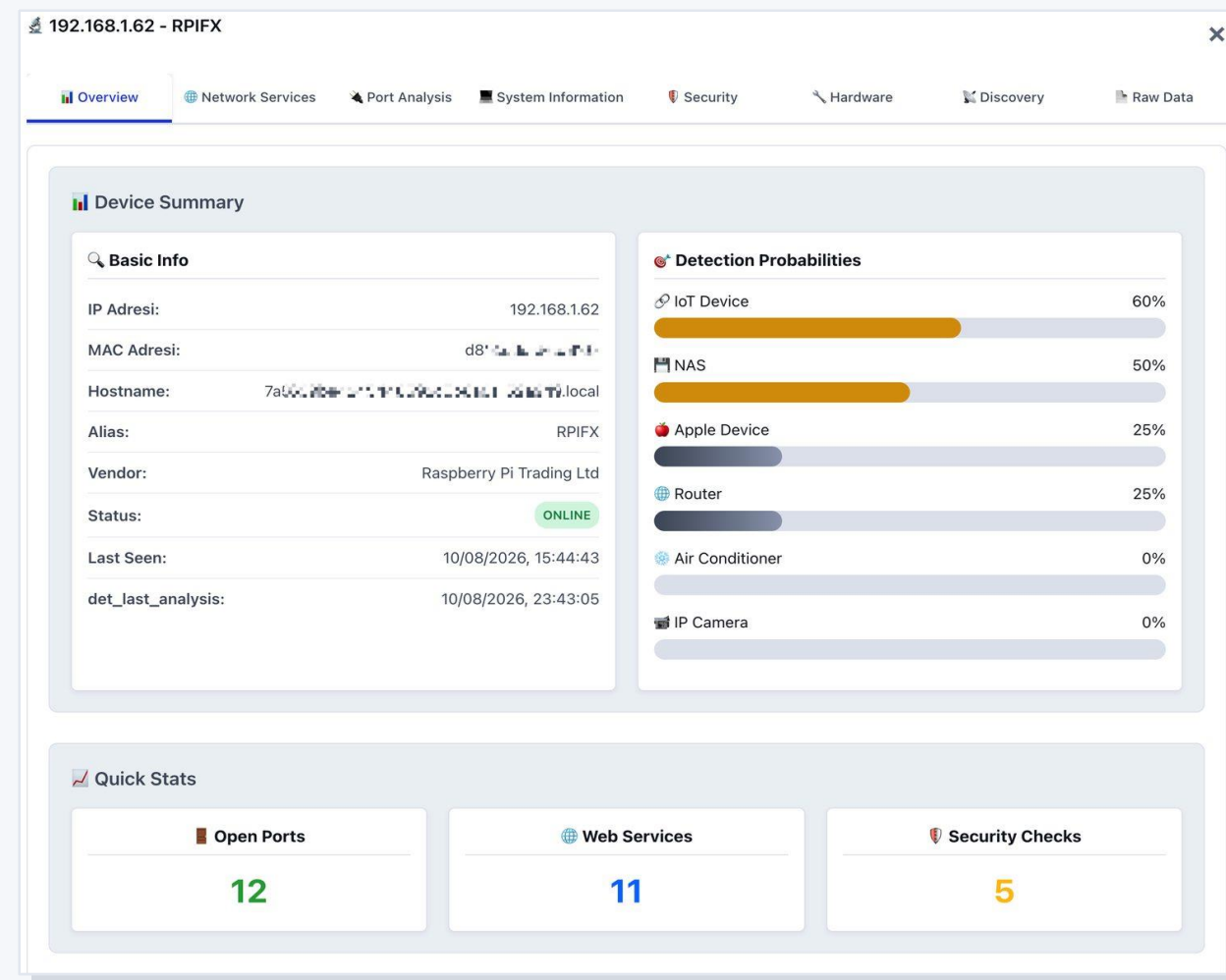




From a MAC address to Device Type and Name.

Identification is the hard half of network scanning. MyNeS stacks every cheap signal first, and only then offers to spend an AI call.

- OUI / vendor — IEEE OUI, MA-M, OUI-36, IAB and CID records, 37,000+ organisations, refreshed from IEEE with an online API fallback.
- Hostname patterns, open-port and service signatures, banner grabs on known to be open.
- Confidence scores and automatic alias suggestions, classified into IoT / Server / Server / Router / Phone and more.
- Identify with AI — send only the collected signals (vendor, OUI, ports, banners) to Anthropic, OpenAI or OpenRouter and get ranked back, each mapped to a real MyNeS device type.
- The button does not appear at all without a key: it is opt-in, not a dependency.
- Honest gap: Fing's crowdsourced fingerprint database is larger. On MyNeS will guess where Fing knows.

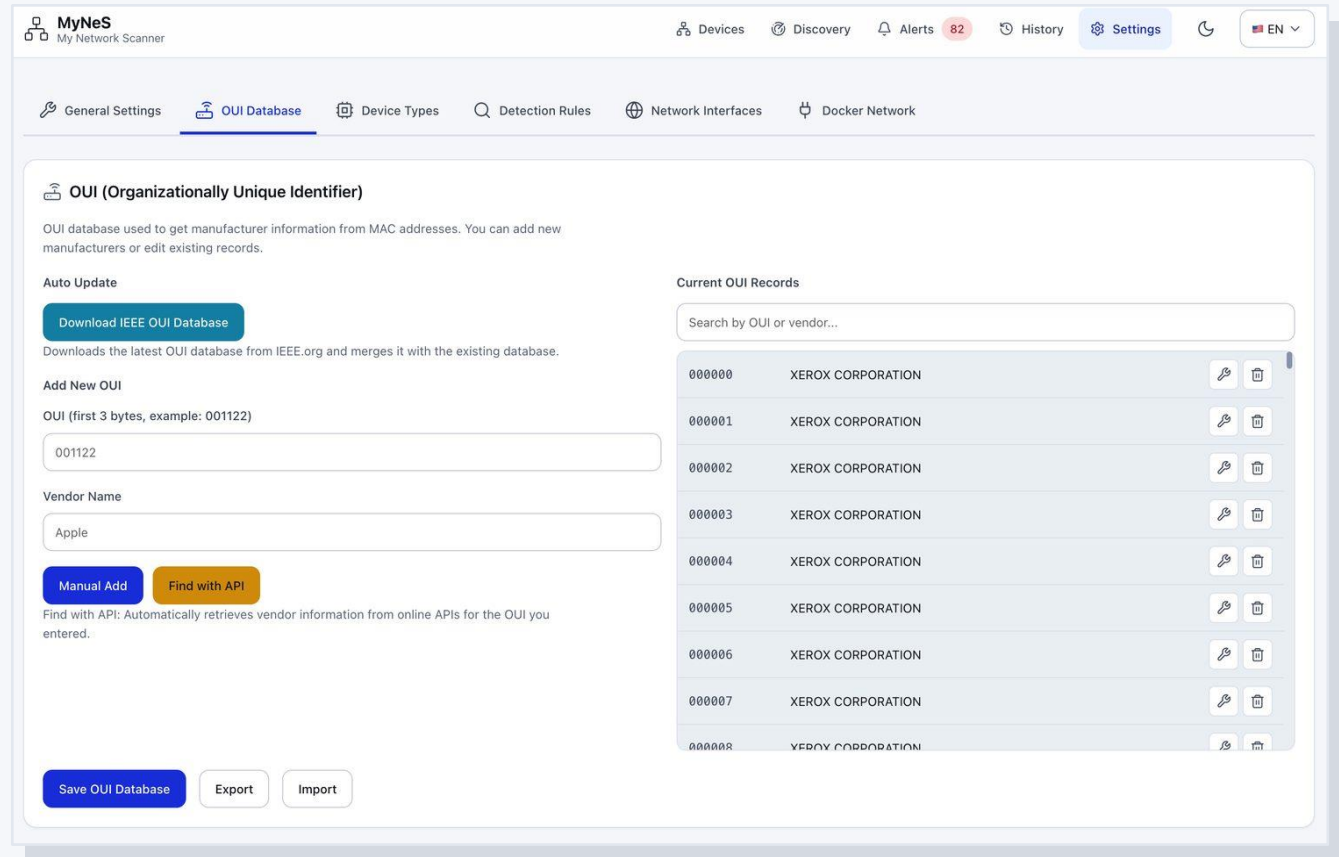


A knowledge base that answers before the LLM does.

mynes-ai is the optional hosted Identify-with-AI backend shared by the web and mobile apps. It tries local knowledge first and only spends an LLM call when that is not enough — so most answers cost nothing.

Step	What it does
1 · OUI DB	Resolve the vendor straight from the MAC
2 · Catalog	Full-text match on any attribute (SQLite FTS5)
3 · Doc RAG	Nearest manual / troubleshooting snippets
4 · LLM	Web search over OUI, manuals and vendor sites
5 · Crawl	Headless fallback when web search is blocked

```
# a confident catalog hit answers with no LLM and no cost
curl -X POST localhost:8182/identify -H 'content-type: application/json' -d '{
  "query_type": "identify",
  "facts": {"mac": "a8:41:f4:..."},
  "oui_vendor": "AzureWave", "open_ports": [80]}
}'
```



The screenshot shows the MyNeS web interface. The top navigation bar includes links for Devices, Discovery, Alerts (82), History, Settings, and a language dropdown (EN). The main navigation bar has tabs for General Settings, OUI Database (active), Device Types, Detection Rules, Network Interfaces, and Docker Network.

The OUI (Organizationally Unique Identifier) section is displayed. It includes a description: "OUI database used to get manufacturer information from MAC addresses. You can add new manufacturers or edit existing records."

Under the "Auto Update" section, there is a "Download IEEE OUI Database" button. Below it, a note states: "Downloads the latest OUI database from IEEE.org and merges it with the existing database."

The "Add New OUI" section contains a form with the following fields:

- OUI (first 3 bytes, example: 001122): A text input field containing "001122".
- Vendor Name: A text input field containing "Apple".

 Below the form are two buttons: "Manual Add" and "Find with API". A note below the buttons states: "Find with API: Automatically retrieves vendor information from online APIs for the OUI you entered."

At the bottom of the section are three buttons: "Save OUI Database", "Export", and "Import".

On the right side, the "Current OUI Records" section is visible. It has a search bar labeled "Search by OUI or vendor...". Below the search bar is a table listing OUI records:

OUI	Vendor	Actions
000000	XEROX CORPORATION	[Edit] [Delete]
000001	XEROX CORPORATION	[Edit] [Delete]
000002	XEROX CORPORATION	[Edit] [Delete]
000003	XEROX CORPORATION	[Edit] [Delete]
000004	XEROX CORPORATION	[Edit] [Delete]
000005	XEROX CORPORATION	[Edit] [Delete]
000006	XEROX CORPORATION	[Edit] [Delete]
000007	XEROX CORPORATION	[Edit] [Delete]
000008	XEROX CORPORATION	[Edit] [Delete]



Scan, diff, and only then interrupt me.

MyNeS re-scans on a schedule, diffs the result against the previous run, and raises only what changed in a way you would want to know about.

- New device joined · device went offline · IP changed
- MAC changed for a known IP — the ARP-spoofing signal.
- A new port opened on a device that did not have it yesterday.
- Low voltage (Raspberry Pi power problems), low battery, high latency, weak signal.
- Channels: MyNeS's own Web Push (no Home Assistant and no third-party service needed), Home Assistant **notify**, ntfy, Telegram, Webhook, Slack, Discord and e-mail.
- Rotating daily logs with a day picker survive a restart, so “what happened at 03:00” is answerable.

The screenshot shows the 'Monitoring & Alerts' section of the MyNeS web interface. At the top, there's a navigation bar with 'Devices', 'Discovery', 'Alerts' (with a red badge showing 82), 'History', 'Settings', and a language selector set to 'EN'. Below the navigation bar, the 'Monitoring & Alerts' title is followed by a subtitle 'Scan on a schedule, spot what changed, get told about it.' and a 'Run scan now' button. The main content area features four summary cards: '82 TOTAL ALERTS', '82 UNREAD', '0 CRITICAL', and 'every 15 min SCHEDULE'. Below these is a status bar showing 'LAST SCAN 8m ago' (8/4/2026, 2:03:39 PM), 'RESULT 39 devices - 2 alerts' (took 93.4s), 'NEXT SCAN in 7 min' (8/4/2026, 2:18:39 PM), and 'STATE running' (Comparing against 39 known devices). The 'Scheduled scanning' section is enabled and shows settings for 'Scan every 15 minutes', 'Declare offline after 3 missed scans', 'Under-voltage threshold (V) 4.7', 'Low battery threshold (%) 20', 'High latency threshold (ms) 500', and 'Extras' including 'Include mDNS / SSDP / BLE sweep' (enabled) and 'Publish to Home Assistant (MQTT)' (disabled). A 'Save schedule' button is at the bottom right of this section. The 'Notifications on this device' section is turned 'On' and includes a 'Send test' button and a message: 'This device is registered. Alerts arrive even with the tab closed.' The 'Notification channels' section at the bottom has an 'Add channel' button.



The whole LAN's exposure, worst first.

One screen for fleet risk and known-CVE exposure, built from a curated offline mapping rather than a live feed.

- Fleet risk score with a severity split — Critical / High / Medium / Low.
- What to fix first — exposed Telnet, SMB/NetBIOS and friends, each with the number of devices affected.
- Per device — scan, acknowledge, watch & alert, export.
- Encrypted credential storage (Fernet / AES-256) for SSH, FTP, SNMP, with restricted file permissions.
- Sensitive data is stripped on export, so a shared inventory file is safe to share.
- The container is hardened: no perl-base, debconf, gosu, curl, pip or setuptools — 70 CVEs Debian had not patched, removed by Runs as uid 1000, never privileged.

MyNeS
My Network Scanner

Navigation: Devices | Discovery | Alerts 99+ | **Security** | History | Settings | EN

Security

Attack surface and known-CVE exposure across every device — worst first. Curated offline match, not a live feed.

[Scan for vulnerabilities](#) [Refresh](#)

28

MEDIUM

Fleet risk posture
Weighted from the worst device plus the spread of at-risk hosts.

36
TOTAL DEVICES

5
AT RISK

0
CVE FINDINGS

9
EXPOSURES

Severity distribution 8 active

Critical 0 High 1 Medium 7 Low 0

What to fix first

- HIGH** **Telnet is open**
Telnet sends credentials in plaintext and has no modern security model. If this is a device admin console, replace it with SSH or the vendor app wherever the firmware allows disabling Telnet.
1 device(s)
- MEDIUM** **SMB/NetBIOS is open**
If this host still has SMBv1 enabled, it is vulnerable to EternalBlue (CVE-2017-0144, the WannaCry/NotPetya vector). Confirm SMBv1 is disabled and only SMBv2/3 is offered.
4 device(s)
- MEDIUM** **VNC is open**
Many VNC servers ship with a weak or no password by default. Confirm a strong password is set and VNC is not reachable from outside the LAN.
3 device(s)

▼ **Per-device detail** 5

Search IP, name, vendor or type... All severities

☐ 0 selected ✓ Acknowledge Un-acknowledge 🔔 Watch & alert 📄 Export 🔄 Re-scan

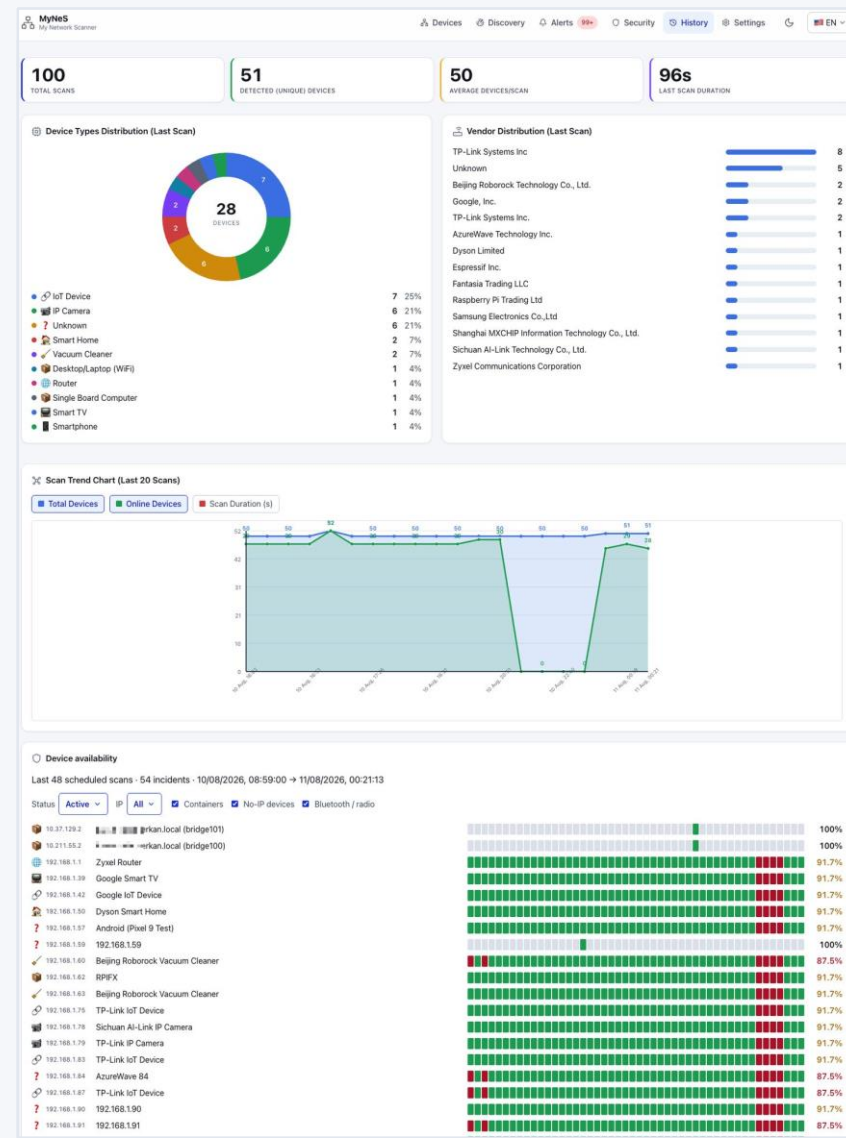
☐	DEVICE	RISK	SCORE ▼	FINDINGS	EXPOSURES	DETAILS
☐	RPIFX 192.168.1.62	MEDIUM	12	0	2	Show details
☐	Sichuan AI-Link IP Camera 192.168.1.78	HIGH	12	0	1	Show details
☐	Raspberry Pi 192.168.1.186	MEDIUM	12	0	2	Show details
☐	Raspberry Pi Single Board Computer 192.168.1.116	MEDIUM	12	0	2	Show details



Your network, over time.

Every scan is kept, so the inventory is a history rather than a snapshot.

- Per-scan results, device counts and statistics you can page back through.
- When did this device first appear? —the question that turns an unknown MAC into a story.
- JSON backup and restore for the whole inventory: aliases, notes, custom types, trust levels.
- Every save snapshots the previous store first (v1.8.1) —a degraded scan can no longer quietly wipe a year of device edits.





A first-class target, not an afterthought. afterthought.

Home Assistant is where most of this inventory wants to end up — so MyNeS speaks its native language.

- MQTT Discovery push — every discovered device becomes an HA entity with no glue code and no template YAML.
- REST / WebSocket pull from HA, so MyNeS can compare what it against HA's own device registry and flag the ones that quietly
- A Home Assistant add-on (mynes-addon, aarch64 + amd64) installs it from from the Add-on Store like any other.
- InfluxDB 2 push and Prometheus scrape for Grafana dashboards.
- A live REST API catalogue read from the running app and downloadable as OpenAPI (swagger.json).
- GET /api/capabilities tells you exactly which discovery methods are working working and why the others are not.

The screenshot shows the 'MyNeS' web interface with the 'Settings' tab selected. The top navigation bar includes links for Devices, Discovery, Alerts (99+), Security, History, Settings, and a language dropdown (EN). The main settings area is divided into three sections:

- InfluxDB 2 (Grafana push)**: A section for pushing scan data to InfluxDB 2. It includes a description: 'Push each scan to InfluxDB 2 as a time series for Grafana dashboards. The token is a secret and is stored server-side only, never in git.' Below this are input fields for 'InfluxDB URL' (http://localhost:8086), 'Organization', 'Bucket', and 'API token' (with a placeholder 'Paste an InfluxDB API token'). There is an 'Enabled' checkbox and 'Save' and 'Test' buttons.
- Prometheus (Grafana scrape)**: A section for scraping metrics from Prometheus or Grafana Agent. It includes a description: 'Point a Prometheus scrape or Grafana Agent at this URL. It needs no authentication, like the health check.' Below this is a 'Scrape URL' input field containing 'http://localhost:5883/api/metrics'.
- REST API catalog**: A section for viewing and downloading the API catalog. It includes a description: 'Every API route this install exposes, read live from the app.' Below this are 'Load catalog' and 'Download OpenAPI (swagger.json)' buttons.

Below the REST API catalog section, there are two lists of API endpoints:

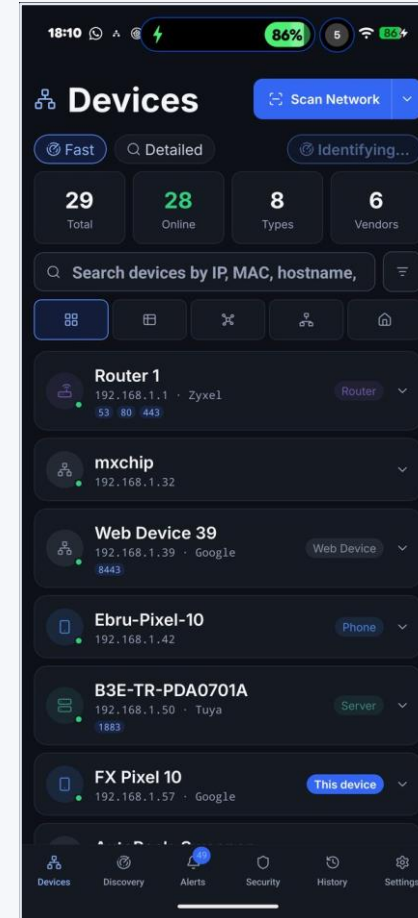
- ALERTS**:
 - GET /api/alerts
 - DELETE /api/alerts
 - POST /api/alerts/read
- AUTH**:
 - POST /api/auth/credentials (with a note: 'Set the login credentials from the Settings page.')
 - DELETE /api/auth/credentials
 - GET /api/auth/status
 - POST /api/auth/status (with a note: 'Turn the login gate on or off.')



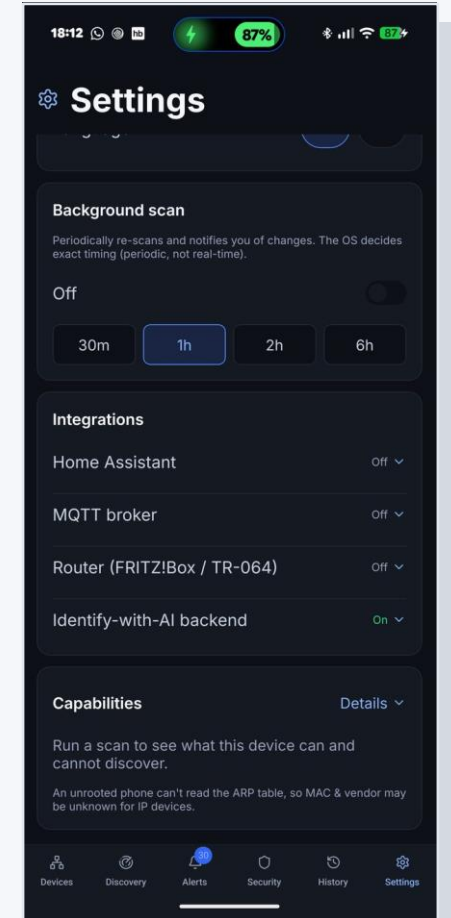
The phone is the scanner.

Not a remote control for a server — a standalone app that runs the whole scanning engine on-device. No server to run, no Docker, no pairing.

- React Native + Expo + TypeScript — one codebase, Android and iOS, captured here on a real iPhone and a Pixel 10 Pro XL scanning a live home network.
- Ping / TCP-connect sweep primes the kernel ARP cache, then the bundled `mynes-ping` native module reads MAC addresses out of it — **without root**. On iOS it reads the route buffer directly (`NET_RT_FLAGS`).
- Vendor offline from the bundled IEEE table; names from reverse-DNS, then and NetBIOS; type from ports refined by banner grabs.
- Design parity is generated, not copied — the web app's `design-system.css` and icon set icon set compile into the mobile theme, so the two never drift.
- The honest constraint: an unrooted phone cannot send raw ARP, so some pure-IP pure-IP devices keep an unknown MAC. The capabilities screen says so rather than hiding it.



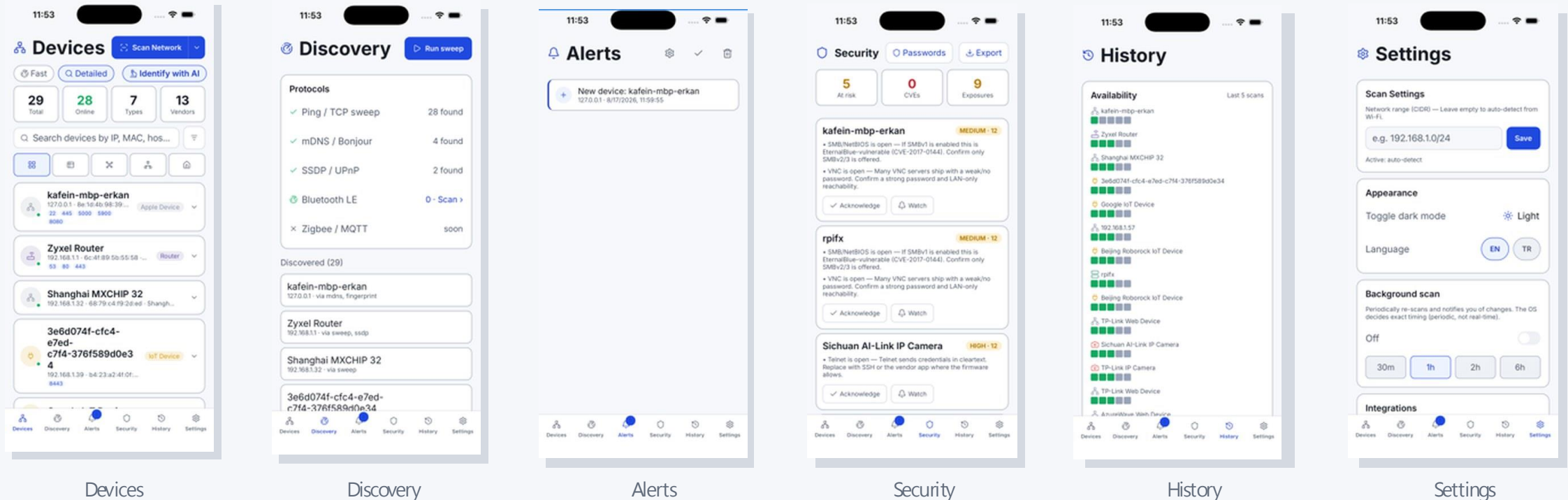
Devices



Capabilities



Everything the web app does, in your pocket.



- Devices—live counters (total / online / types / vendors), search, and the same card, table, graph, topology and floor-plan switches as the web app.
- Discovery · Alerts · Security · History · Settings—the same five pillars, laid out for a

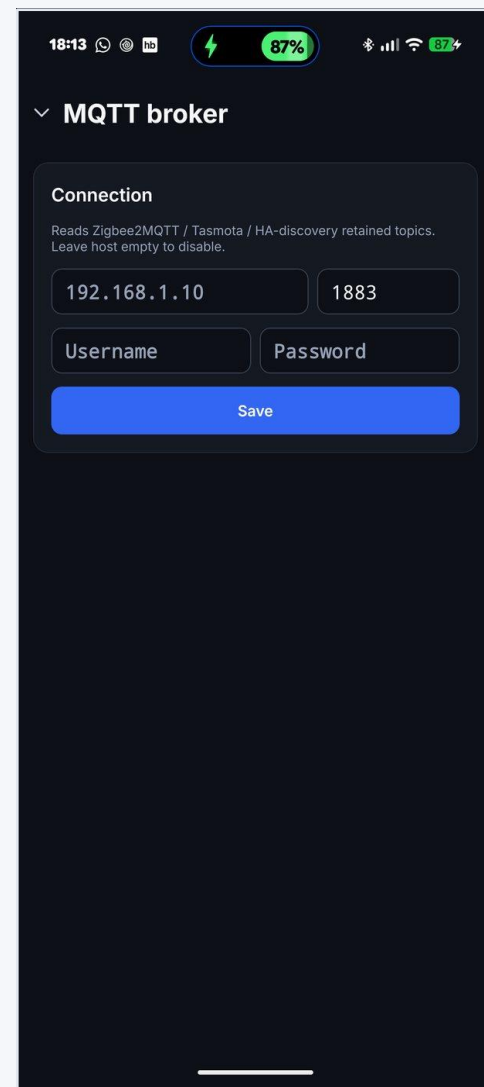
- Light and dark follow the OS; the interface ships in Turkish and English.



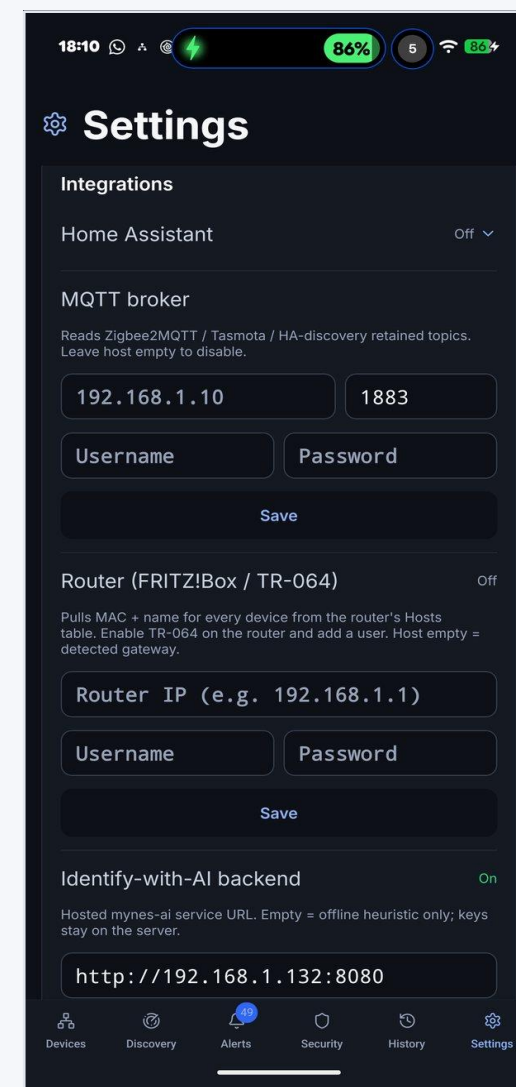
The same integrations, from the phone.

The mobile app carries the settings the web app has — not a reduced subset.

- MQTT broker — point it at Zigbee2MQTT / Z-Wave JS / Tasmota and the radio the radio devices appear on the phone too.
- Home Assistant and Router (FRITZ!Box / TR-064) — pull MAC and name for every device from the router's own hosts table.
- Identify-with-AI backend — point it at your own mynes-ai instance, or leave it empty for the offline heuristic. Provider keys stay on the server, never on the phone.
- Background scan every 30 m / 1 h / 2 h / 6 h, with OS-decided timing notification when something changes.
- Scan range auto-detected from Wi-Fi, or set a CIDR by hand.
- Dark mode and EN / TR switch right in Settings.



MQTT broker



Integrations



MyNeS is not one app — it is a small ecosystem.

Each piece is independently useful, and each one is optional except the first.

Core

my_network_scanner

The scanner, the web UI and the API. Python + Flask, Flask, AGPL-3.0, published to Docker Hub for amd64 and arm64.

Mobile

mynes-mobile

Standalone Android and iOS app running the scan engine on-device. React Native + Expo + TypeScript. TypeScript.

AI

mynes-ai

Optional hosted Identify-with-AI backend. Local catalog catalog + RAG first, LLM only only when needed. Keys never never leave it.

Home Assistant

mynes-addon

Add-on repository for Home Home Assistant. Install from from the Add-on Store; aarch64 and amd64.

Casa OS

mynes-casaos-store

Third-party app store source source for CasaOS and ZimaOS, so MyNeS installs with one click there too.



Persona 01

Homelab / Self-hoster

Runs a Pi, a NAS or a mini-PC with a dozen containers on it. Wants to know exactly what is on the LAN, and wants it to stay on the LAN.

The pain today

- Fing wants an account, and the inventory lives on someone else's servers.
- Angry IP Scanner is a one-shot tool: no history, no naming, nothing persists between runs.
- A dozen containers on three Docker networks show up show up as mystery IPs with no context.
- No record of when a device first appeared or when a when a port opened.
- Grafana is already running, but the LAN is the one thing one thing not in it.

What they use

- Docker Compose with `network_mode: host` plus `NET_RAW` and `NET_ADMIN` — the minimum that lets raw ARP and multicast actually see the LAN.
- `GET /api/capabilities` — says which discovery are live and why the rest are not, instead of finding fewer devices.
- Docker awareness — containers, virtual networks and IP and IP assignments, nested under the host that runs runs them.
- InfluxDB 2 push / Prometheus scrape straight Grafana that is already up.
- ntfy, Telegram, Discord, Slack or a webhook for alerts; alerts; the full REST API for everything else.

What they get

- AGPL-3.0, one container, no account and nothing phoning home.
- `linux/amd64` and `linux/arm64` — the Pi in the cupboard is enough.
- Runs as `uid 1000`, **never privileged**, on a base stripped of 70 unpatched CVEs.
- Scan history and rotating daily logs that survive a restart.
- A device store that snapshots itself before every save, so a bad scan is one file-copy away from undone.



Persona 02

Smart-home owner / Home Assistant user

Has forty devices in Home Assistant and no idea how many more are on the network. Half the house talks Zigbee, and no scanner has ever shown any of it.

The pain today

- Zigbee bulbs and Z-Wave locks are invisible to every IP scanner they have tried.
- A device stopped responding last week and nobody noticed until the automation failed.
- HA's device registry and the actual network have quietly drifted apart.
- Adding a new device to HA means hunting for its IP by IP by hand.
- NetAlertX finds the IP devices, but not the radio ones.

What they use

- MQTT discovery — point MyNeS at the Zigbee2MQTT / Z-Wave JS Z-Wave JS broker that is already running and the radio devices land in the inventory.
- Matter over mDNS (`_matter._tcp`) and Bluetooth LE advertisements for the rest.
- MQTT Discovery push — every device becomes an HA entity entity automatically, no template YAML.
- REST/ WebSocket pull from HA to compare found devices against HA's registry and flag the ones that unplugged.
- The Home Assistant add-on (mynes-addon) so it installs Add-on Store like anything else.
- The floor plan — drag each device into the room it is actually in.

What they get

- The devices that were never on the network map are on it now.
- Offline detection that fires **before** an automation breaks.
- One inventory that HA and a human can both read.
- Alerts through HA's own **notify** service — or MyNeS's Web Push, with no HA in the loop at all.
- **Named dependency:** Zigbee/Z-Wave visibility an existing MQTT broker. MyNeS reads its topics; it does not drive a radio.



Persona 03

End user / the family

Not a DevOps engineer and will never run Docker. Wants one honest answer to one question: who is on my Wi-Fi right now?

The pain today

- The router's admin page lists twenty MAC addresses and names none of them.
- "Is that unknown device the neighbour, or my own own robot vacuum?"
- Every friendly app wants an account and puts the house house inventory in someone's cloud.
- There is no server at home and there is never going to be one.
- Most of these tools are English-only.

What they use

- The mobile app — install, open, scan. The phone runs the engine; there is no server to set up at all.
- Card view with the plain-language summary: total, total, online, types, vendors.
- Trust levels — mark each device Known, Trusted, then filter for what is left.
- Background scan every hour, with a notification when a when a new device joins.
- Wake-on-LAN to bring the desktop up from the
- Or, on a shared computer: the PWA — add the web app to the home screen, no store needed.

What they get

- An answer in a minute, with no account and no cloud.
- Every device with a real name, a type and a room— because you named them once.
- The interface in Turkish, throughout, not a partial translation.
- Light and dark follow the phone; the layout works from 320px to a TV.
- **Honest limits:** an unrooted phone cannot read every every MAC, so some devices stay partly unknown —and —and the app says which.



Self-hosted Fing that also sees the devices with no IP address.

Types: partial ok no only MyNeS

Capability	MyNeS	Fing	Angry IP Scanner	Advanced IP Scanner	NetAlertX	nmap / Zenmap
License & cost	AGPL-3.0 · free	Proprietary · free app, paid box/tiers	GPL-2.0 · free	Proprietary · freeware	GPL-3.0 · free	NPSL · free
Account required	✗ none	✓ for most useful features	✗	✗	✗	✗
Cloud / telemetry	✗ none	central to the product	✗ none	~ update check	✗ none	✗ none
Runs as	Self-hosted web app	Mobile + desktop + hardware	Desktop app	Windows desktop	Self-hosted web app	CLI (+ Zenmap)
ARP layer-2 discovery	✓ raw + fallback	✓	✗ ping + port	✓	✓	✓
mDNS / SSDP / UPnP	✓	✓	✗	~	~	~ scripted
Matter	✓ _matter._tcp	~	✗	✗	✗	✗
Bluetooth LE ★	✓	~ mobile only	✗	✗	✗	✗
Zigbee / Z-Wave via MQTT ★	✓	✗	✗	✗	✗	✗
Docker containers & networks ★	✓	✗	✗	✗	✗	✗
Monitoring + alerts	✓ 8 rule types	~ box / premium	✗	✗	✓ its core	✗
Home Assistant integration	✓ MQTT Discovery + add-on	✗	✗	✗	~ community	✗
Views	Card, table, graph, topology, floor plan plan	List	List	List	List	~ Zenmap topology
Native mobile app	✓ iOS + Android + PWA	✓ native	✗	✗	~ PWA	✗
Device fingerprint depth	~ OUI + heuristics + AI	✓ crowdsourced, largest	✗	✗	~	✓ best in class
Maturity	Young · 2025— · one maintainer	~15 years · a company	~20 years	~15 years	Several years, active	~25 years



Where MyNeS wins — and where you should use something else.

Where it wins

Where it doesn't — reach for something else

- It sees what nothing else sees. Bluetooth LE, Zigbee, Z-Wave and Matter are invisible to every other tool in the matrix. In a matrix. In a 2026 smart home that is often a third of the devices.
- Self-hosted, no account, no telemetry. AGPL-3.0, one container, on hardware you already own.
- Home Assistant is native — MQTT Discovery push, REST pull, and an official add-on repository.
- Graph, topology and floor-plan views that no competitor in the category ships.
- Docker-aware — containers and their virtual networks, nested under their host.
- It tells you what it could not do — /api/capabilities instead of silently finding fewer devices.

- Device fingerprinting. Fing's crowdsourced database is far larger; obscure hardware stays unknown more single biggest honest gap.
- WAN-side tooling. No internet speed test, no ISP outage detection, no bandwidth analysis. → Fing / Fingbox.
- Enforcement. MyNeS observes; it does not block a device or shape traffic. → Fingbox, or your router.
- One-tap install. Fing is a tap; MyNeS is a container, host networking and a capabilities conversation.
- Big networks. Python and Flask on a /16 is minutes, not seconds. This is a home LAN tool, not a campus one.
- Maturity. Young project, one maintainer, no decade of hardening. → NetAlertX for battle-tested presence

If you need...	Reach for
A self-hosted device inventory that also sees Zigbee, Z-Wave, BLE and Matter	MyNeS
Every device in Home Assistant with no glue code	MyNeS
A scanner on a phone with no server anywhere	MyNeS Mobile
A speed test, ISP monitoring or network-level blocking	Fing / Fingbox
Deep port and service interrogation	nmap



Five ways in — pick the one that matches your house.

The container is the reference install. The add-on and the CasaOS source are the same image with the setup already done; the phone needs no server at all.

Docker · the reference

```
# host networking is what lets it see the LAN
docker compose up -d

# then open
http://<host-ip>:5883
```

Home Assistant add-on

```
# Settings → Add-ons → Store → : → Repositories
github.com/fxerkan/mynes-addon

# install, start, open on port 5883
```

CasaOS / ZimaOS

```
# App Store → : → Add Source
https://cdn.jsdelivr.net/gh/fxerkan/
my nes-casa-os-store@gh-pages/store/main.zip
```




My Network Scanner

MyNeS

An IP scan shows you addresses.
addresses. MyNeS shows you your
your house.

Self-hosted. No account. No cloud. And it finds the devices that were never on the map.

[docker pull fxerkan/my_network_scanner](#)

AGPL-3.0

amd64 · arm64

Home Assistant add-on

CasaOS / ZimaOS

iOS + Android

- [github.com/fxerkan/my_network_scanner](#)
- Docker Hub
- mynes-mobile
- mynes-ai
- HA add-on

